Constitutional Semantic Engineering

A Discipline for Software Whose Meaning Must Survive Execution, Refactor, Migration, and Governance

The code may run; the question is whether its meaning can still stand.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Software systems often remain operational long after their meaning has begun to decay. Tests may pass while authority assumptions disappear into helper functions; non-binding traces may accumulate while evidence remains inadmissible; documentation may describe an earlier standing while current artifacts silently imply another. Existing software practices address parts of this problem. Domain-Driven Design protects domain language and bounded contexts. Hexagonal architecture separates application cores from external devices and databases. Functional-core patterns separate deterministic logic from impure effects. Event sourcing and provenance systems support reconstructability. Architecture Decision Records preserve design rationale. Capability systems sharpen authority and least privilege. Yet these practices are rarely unified into a discipline that treats meaning, authority, evidence, refusal, standing, and non-claims as first-class software artifacts.

This paper proposes Constitutional Semantic Engineering (CSE): a software-engineering discipline for systems whose behavior must not merely execute, but remain semantically admissible across change. CSE introduces a standing-aware model of software development in which transitions are explicitly authorized, effects are mediated, receipt-bound artifacts serve as evidence, refusals are positive governance events, and claims are constrained by written non-claims. Its purpose is to prevent semantic debt, especially semantic locality debt: the cost incurred when meaning exists somewhere in a system but not in a stable, canonical, recoverable home.

CSE is organized across three layers. First, conceptual axioms define the discipline: semantic locality, explicit authority, evidence-bound claims, refusal as success, scoped standing, written non-claims, and semantic refactor discipline. Second, canonical governance artifacts make those axioms operational: receipts, policies, standing records, claim ledgers, migration contracts, glossaries, and non-claim surfaces. Third, machine-checkable invariants allow tooling to inspect

whether a package preserves effect separation, authority completeness, receipt binding, refusal totality, non-claim preservation, standing monotonicity, replay determinism, and anti-self-ratification: the requirement that protected standing never rest on an evidentiary route wholly controlled by the subject whose standing is at issue.

CSE does not replace established practices. It constitutionalizes them. It treats ports as authority boundaries, events as evidence candidates, tests as insufficient without claim discipline, and refactors as potentially semantic operations. The resulting capability is not simply cleaner code, but governable software: software whose meaning can be admitted, refused, replayed, migrated, audited, and compared without semantic archaeology.

1. Introduction: Software Can Work While Its Meaning Decays

A software system can continue to run while losing the meaning needed to govern it. This is not a paradox. Operational success and semantic admissibility are different properties.

A test suite may confirm that output remains unchanged while no longer revealing why that output is authorized. A database migration may preserve fields while losing the distinction between validation, admission, execution, and standing. A refactor may reduce code length while merging policy, effect, and authority into a single convenience function. A runtime trace may record that something happened while failing to bind the event to the state, policy, artifact, and authority that would make the event admissible as evidence.

This paper begins from a simple observation:

> Software does not only accumulate technical debt. It accumulates semantic debt.

Technical debt traditionally concerns the future cost of suboptimal implementation decisions. Kruchten, Nord, and Ozkaya describe the debt metaphor as an attempt to organize the landscape of software-development compromises and their future consequences (Kruchten, Nord, and Ozkaya, 2012). More recent discussions have extended the frame toward cognitive and intent debt: the erosion of shared understanding and the loss of captured rationale needed by humans and machines to safely evolve systems (Storey, 2026). Semantic debt is adjacent but distinct. It concerns not merely code quality, team understanding, or documented intent, but the recoverability and admissibility of meaning itself.

A system has semantic debt when its names, categories, boundaries, procedures, authority assumptions, evidence, or claims become underdefined, contradictory, stale, fragmented, opaque, overcoupled, or overclaimed. The system may still compile. It may still ship. It may still pass tests. But it has lost some of the structure needed to answer: what does this behavior mean, who or what

authorized it, what standing does it have, what evidence supports it, and what remains unclaimed?

Constitutional Semantic Engineering is proposed as a discipline for preventing and repaying that debt.

2. Definition

Constitutional Semantic Engineering is a software-engineering discipline for building systems whose meaning, authority, effects, evidence, refusals, standings, and non-claims remain explicit and recoverable across execution, refactor, migration, automation, and governance.

In shorter form:

> CSE is engineering for software whose behavior must remain semantically admissible.

The word constitutional is used deliberately. A constitution is not merely a document of aspirations. It defines authority, boundaries, procedures, permitted transitions, refusals, and claims that may not be made without standing. In CSE, software is modeled not only as executable behavior but as a governed semantic system. Its transitions must be admissible. Its effects must be authorized. Its evidence must be receipt-bound. Its claims must not outrun its receipts.

The discipline can be summarized by seven commitments:

1. Meaning must have locality. Every meaningful distinction should have a stable home.
2. Authority must be explicit. Effects and transitions are not justified merely because code can perform them.
3. Evidence must bind claims. Non-binding traces are not enough; claims require structured, replayable support.
4. Refusal is a success condition. A correctly refused transition preserves the system.
5. Standing must be scoped. Validation, admission, execution, standing, equivalence, and self-hosting are different statuses.
6. Non-claims must be written. A system must say what it does not yet prove.
7. Refactor must preserve semantics, not only behavior. A cleanup that destroys recoverable meaning creates semantic debt.
8. Evidence for protected standing must not be wholly self-controlled. A system must not be able to establish its own high-consequence standing from evidence it alone produces, alters, suppresses, or selects.

3. Formal Model

This section gives a minimal calculus for CSE. The purpose is not to replace programming languages, proof assistants, or runtime systems. The purpose is to

make the central discipline mechanically checkable.

Let:

- S be system state.
- Ctx be governed context, including authority references, policy version, capability set, time source, and replay parameters.
- Cmd be a command.
- S' be successor state.
- R be a receipt.
- Eff^* be a sequence or set of effect requests and effect decisions.
- Ref^* be a sequence or set of refusals.
- st be current standing.
- NC be the active set of non-claims.

3.1 Transition Judgment

A governed transition has the form:

$$\begin{aligned}&\langle S, Ctx,\\ &Cmd\rangle \overset{auth,pol}{\Longrightarrow} \langle S',\\ &R, Eff^*, Ref^*\rangle\end{aligned}$$

This states that, under explicit authority and policy, a command applied to a prior state and governed context produces successor state, receipt, effects, and refusals.

The judgment is not merely an execution trace. It is a semantic transition claim. It can be checked for policy totality, authority completeness, receipt binding, effect mediation, refusal totality, and standing validity.

3.2 Receipt Admissibility Predicate

A receipt is admissible only if it binds the relevant transition fields and violates no governing invariant:

$$\begin{aligned}&\mathsf{adm}(R)\\ &\quad\iff \mathsf{bind}(R.S_{prev}, R.S_{next}, R.auth, R.pol, R.hashes)\\ &\wedge\\ &\neq g\mathsf{viol}(R)\end{aligned}$$

The predicate bind requires the receipt to bind at least prior state, next state, authority reference, policy version, and relevant artifact hashes. The predicate viol detects invariant failure, unsupported standing transition, missing authority, unauthorized effect, schema mismatch, or contradiction with active non-claims.

3.3 Standing Transition

Standing evolves by a governed standing-transition function:

$$st' = \delta_{st}(st, R)$$

The transition is constrained by monotonicity unless an explicit revocation or downgrade record exists. In ordinary ascent, standing cannot increase by implication. It must be granted by an admissible receipt whose operation and policy entail that standing transition.

For example:

```
validated
  does not imply admitted

admitted
  does not imply governed execution standing

governed execution standing
  does not imply schema-conformant receipt standing
```

3.4 Claim Rule

A claim k is admissible only when admissible receipts entail it and active non-claims do not contradict it:

$$\begin{aligned}
&\mathsf{claim}(k) \\
&\Rightarrow \exists R_1..R_n : \left(\bigwedge_i \mathsf{adm}(R_i) \right) \\
&\wedge \mathsf{entails}(R_1..R_n, k) \\
&\wedge \\
&\neq g\mathsf{contradicted}(k, NC)
\end{aligned}$$

This is the constitutional center of CSE: claims must be earned. A system cannot merely assert standing, equivalence, execution authority, or self-hosting. It must exhibit admissible receipts that entail the claim and must not be blocked by active non-claims.

3.5 What the Formal Model Adds

The formal model turns CSE from a conceptual discipline into a checkable discipline. It provides:

- a bridge from prose claims to governance artifacts;
- a way to evaluate refactoring and migration as semantic operations;

- a basis for automated claim evaluation;
- a method for distinguishing validation, admission, execution, and standing;
- a structure for blocking unsupported claims through active non-claims.

The formal model is intentionally minimal. Its value is not in mathematical ornament. Its value is in giving tools something exact to check.

4. Canonical Governance Artifacts

CSE requires canonical artifacts for receipts, policies, standing records, claims, glossaries, and non-claims. These artifacts are normative within a CSE package because they define the surfaces by which meaning becomes inspectable.

4.1 Receipt v0

Receipt v0 is a canonical, content-addressed structure. All fields should be serialized canonically, hashed, and included in the relevant evidence chain.

Required fields:

```
rid
schema
op
ts
S_prev
S_next
in_hash
art_hashes
auth_ref
pol_ver
cap_set
eff_req
eff_adm
eff_ref
ref_codes
nc_refs
st_prev
st_next
sigs
```

Where:

- `rid` is the receipt identifier.
- `schema` is the receipt schema version.
- `op` is the operation.
- `ts` is a timestamp or timestamp placeholder; replay comparison may normalize it.
- `S_prev` is the prior-state hash and optional Merkle root.
- `S_next` is the next-state hash and optional Merkle root.

- `in_hash` is the command or input hash.
- `art_hashes` are relevant artifact hashes.
- `auth_ref` is the authority reference.
- `pol_ver` is the policy version.
- `cap_set` is the capability set in force.
- `eff_req` is the set of effect requests.
- `eff_adm` is the set of admitted effects.
- `eff_ref` is the set of refused effects.
- `ref_codes` are typed refusal codes.
- `nc_refs` are non-claims in force.
- `st_prev` is prior standing.
- `st_next` is next standing.
- `sigs` are optional attestations or signatures.

A receipt is not required to contain all payload content directly. It may bind content by hash, Merkle root, or content-addressed reference. But the binding fields must be sufficient to support replay, audit, and claim evaluation.

A non-binding trace says something happened. A receipt says what happened in a form that can support or refuse a claim.

4.2 Policy Surface

A policy surface is total, versioned, deterministic, side-effect free, and diffable across versions.

The minimal policy form is:

$$\pi : (Cmd, S, Ctx) \to \{permit, refuse(code)\}$$

A policy must not fall through silently. Every command, state, and context triple must either be permitted or refused with a typed refusal code.

Policy versions must be diffable because policy changes are semantic changes. A policy diff is not merely a code diff. It may widen authority, narrow authority, change refusal behavior, change standing ascent, or alter admissible claims.

4.3 Standing Record

A standing record binds a standing transition to an admissible receipt. It should include:

```
standing_record_id
subject_ref
subject_hash
st_prev
st_next
transition_op
```

```
receipt_ref
policy_ref
authority_ref
scope
revocation_terms
nonclaims_preserved
```

Standing records prevent status drift. They make it impossible, or at least mechanically detectable, for validation to become admission, admission to become execution standing, or fixture standing to become broad standing by implication.

4.4 Glossary

A glossary is not decorative documentation. It is part of the semantic surface. A CSE glossary should define:

- acronyms;
- standing terms;
- receipt terms;
- policy terms;
- refusal codes;
- claim names;
- non-claim names;
- deprecated names;
- migration mappings for renamed concepts.

No new verifier acronym, standing term, claim kind, refusal code, or policy category should be considered name-complete until entered into the glossary.

4.5 Non-Claims Surface

A non-claims surface records what the system does not assert. It should include:

```
nonclaim_id
blocked_claim
reason
scope
introduced_by
active_since
revocation_conditions
revoked_by
```

Non-claims persist until revoked by admissible record. A missing non-claim is not proof that a claim is admissible; it is merely absence of an explicit block. But deletion of an active non-claim without revocation is a semantic violation.

5. Machine-Checkable Invariants

CSE can be supported by machine-checkable invariants. These invariants are intentionally lightweight. They can be checked by static analysis, schema validation, fixture replay, or specialized tools.

5.1 Effect Separation

All effects must occur through adapters or mediated effect surfaces.

$$\forall t,\ \mathsf{Eff}(t) \subseteq adapters$$

The deterministic core may request effects. It must not perform unauthorized world-touching operations directly.

5.2 Authority Completeness

Every admitted effect must have an authority reference.

$$\forall t,\ \mathsf{Eff}_{adm}(t) \Rightarrow auth_ref(t) \neq \varnothing$$

Ability to perform an effect is not authority to perform it.

5.3 Receipt Binding

Every transition receipt must bind prior and successor state.

$$\forall t,\ hash(S_{prev}), hash(S_{next}) \in R$$

A transition that does not bind its prior and successor states cannot support a strong standing or migration claim.

5.4 Refusal Totality

The policy surface must be total. There must be no implicit fallthrough.

$$\forall Cmd, S, Ctx,\ \pi(Cmd, S, Ctx) \in \{permit, refuse(code)\}$$

A missing case is not permission. It is an invalid policy surface.

5.5 Non-Claim Preservation

Non-claims persist unless explicitly revoked by an admissible record.

$$NC_{t+1} \supseteq NC_t \setminus \{nc \mid \mathsf{revoked}(nc, R) \wedge \mathsf{adm}(R)\}$$

This prevents non-claims from silently disappearing during refactor, migration, or standing ascent.

5.6 Standing Monotonicity

Standing does not ascend without a standing-transition record.

$$\begin{aligned} & st_{t+1} \geq st_t \\ & \Rightarrow \exists R : \mathsf{adm}(R) \\ & \wedge \delta_{st}(st_t, R) = st_{t+1} \end{aligned}$$

The symbol $\geq$ here denotes an ordering in a standing graph, not ordinary numeric comparison. If the standing graph is partial rather than total, the invariant applies along declared ascent edges.

Downgrade, revocation, and quarantine are permitted only by explicit record.

5.7 Replay Determinism

Core replay over fixtures must yield identical receipts modulo explicitly normalized fields such as timestamps.

$$Replay(core, fixture)_1 \equiv Replay(core, fixture)_2 \pmod{Norm}$$

The normalization set $Norm$ must be explicit. Hidden nondeterminism is semantic debt.

5.8 Anti-Self-Ratification

Protected standing must not be obtainable through an evidentiary route lying wholly under the control of the subject whose standing is being decided.

Let Θ be a declared control model, let $ctl_\Theta(x, e)$ hold when actor x can unilaterally create, alter, suppress, substitute, or select evidence item e, and let $\mathsf{suff}(E')$ hold when support set E' is sufficient for the requested standing under a fixed adjudicative context.

$$
\begin{aligned}
&\mathsf{asr}(t) \\
&\iff \\
&\neq xists E' \subseteq E_t^+ : \mathsf{suff}(E') \\
&\wedge \forall e \in E',\ ctl_\Theta(subj(t), e)
\end{aligned}
$$

The quantifier is existential and it is the whole content of the invariant. A weaker rule would require that the winning evidence merely include an independent element. This rule forbids the existence of any self-controlled sufficient route, even when an independent route also exists. A system that keeps an apparently independent path while leaving a self-ratifying bypass available does not satisfy this invariant.

Two consequences follow for tooling.

First, independence is a control relation rather than a location. Separate files, processes, containers, model invocations, or organizational names do not establish independence if the subject controls the inputs, configuration, executable, signing key, or output selection of the nominally separate source. The declared Θ is what makes ctl_Θ decidable, and an independence rank over genuinely distinct control domains is what prevents nominal plurality from counting as independence.

Second, the invariant is checked by ablation rather than by inspection. Remove a claimed independent element from the support set and re-run the adjudication. If standing survives, that element was ceremonial rather than load-bearing.

Exhaustive search over support subsets is exponential in the size of the evidence bundle. A conforming implementation must therefore distinguish a bounded check from an exact one, and must report a bounded result as bounded. Where an exact check is required and infeasible, the correct outcome is not-established rather than an approximate pass.

This invariant is the one that most directly protects the claim ledger of Section 7 and the anti-overclaim posture of Section 14.3. Without it, a package can satisfy every other invariant in this section and still promote itself to its highest standing on receipts it alone produced.

6. Static and Dynamic Checks

CSE supports two broad families of checks.

6.1 Static Checks

Static checks inspect the package without necessarily executing the governed transition.

Examples:

- policy totality;

- authority plumbing;
- adapter isolation;
- absence of world-touching calls in core logic;
- glossary coverage;
- non-claim coverage;
- standing graph validity;
- schema completeness;
- claim vocabulary consistency;
- package manifest consistency;
- forbidden import or subprocess usage in core;
- policy diff classification;
- migration contract completeness;
- control-model declaration coverage for evidence sources.

6.2 Dynamic Checks

Dynamic checks execute fixture replay, verifier lanes, or controlled transitions.

Examples:

- receipt schema conformance;
- replay equivalence;
- refusal coverage;
- claim-to-evidence satisfiability;
- policy-permit and policy-refuse behavior;
- standing transition validation;
- receipt diff stability;
- artifact hash consistency;
- archive hygiene;
- evidence ablation for protected standing transitions.

6.3 Example Claim Check

Given the claim:

`export_admitted`

A checker may verify:

$$\begin{aligned}&\exists R : R.op = export\\ &\wedge R.eff_adm\\ &\neq \varnothing\\ &\wedge \pi = permit\\ &\wedge \mathsf{adm}(R)\end{aligned}$$

If no such receipt exists, the claim is inadmissible. If an active non-claim blocks it, the claim is also inadmissible, even if a superficially supporting trace exists.

7. Claim Ledger

CSE benefits from a first-class claim ledger: a graph connecting claims, receipts, non-claims, and derivations.

Let:

- claims be nodes k;
- receipts be nodes R;
- non-claims be nodes NC;
- support edges have the form $R \to k$;
- blocking edges have the form $NC \dashv k$;
- derivation edges have the form $R \to R'$.

The admissible claim frontier is:

$$\mathcal{K}_{adm} = \{k \mid \mathit{supports}(k) \wedge \neg \mathit{blocked}(k)\}$$

The claim ledger allows a system to query what it may currently assert. It also allows auditors, agents, or admission tools to detect claim inflation.

Example questions:

- What claims are supported by admissible receipts?
- Which claims are blocked by active non-claims?
- Which receipts derive from which prior receipts?
- Which claims would become admissible if a specific non-claim were revoked by record?
- Which standing transitions lack admissible support?
- Which supported claims have a sufficient support route lying wholly within a single control domain?

The claim ledger is the memory of what the system is allowed to say.

8. Validation, Admission, Execution, and Standing

CSE reserves four terms that are often blurred in ordinary software practice.

Validation means syntactic or semantic checking. A validator may establish that an artifact conforms to a schema, passes a test, satisfies a fixture, or obeys a local invariant. Validation does not by itself admit the artifact.

Admission means a policy permits an artifact, command, transition, or effect request within a defined scope. Admission is a governed decision, not a synonym for test success.

Execution means effects have been admitted and performed, or that a runtime transition has occurred under a defined execution authority. Execution is not implied merely by admission.

Standing means a governed status that may persist beyond a single check or event. Standing is granted by explicit standing transition rules and remains scoped by those rules.

In short:

```
validation != admission
admission != execution
execution != standing
standing != broad maturity claim
```

This distinction is central to CSE. Many semantic failures arise when one of these terms is used to smuggle another.

9. The Problem CSE Addresses

Modern software practice already knows how to make code more modular, testable, observable, and maintainable. Yet many systems still suffer from meaning drift. CSE names several recurring failure modes.

9.1 Behavior Without Standing

A package can pass tests without being admitted. It can be admitted without having governed execution standing. It can execute without producing schema-conformant receipts. It can produce receipts without earning equivalence, self-hosting, or maturity standing.

When these statuses collapse into a single vague notion of "works," software becomes semantically unsafe. The system can no longer distinguish validation from admission, admission from execution, execution from standing, or standing from broad claim.

9.2 Effects Without Authority

Ordinary programs often treat side effects as implementation details. A function writes to disk, reads the environment, opens a network connection, launches a subprocess, or mutates a database because it has the technical ability to do so.

CSE treats this as incomplete. Every effect-bearing operation should be represented as an authority-bearing act. The question is not merely "can this function write?" but "under what authority does it request or perform the write, and what evidence records that authority?"

Capability-based security and object-capability systems already provide important foundations for authority discipline. Miller, Yee, and Shapiro argue that capability systems provide strong support for least-privilege operation and avoiding confused-deputy problems (Miller, Yee, and Shapiro, 2003). CSE borrows this instinct but applies it semantically: authority is not only a security primitive but a meaning primitive.

9.3 Non-Binding Traces Without Evidence

A system may have logs, metrics, traces, test reports, and screenshots. CSE does not treat these as evidence by default. They are non-binding traces until they are bound into receipts, manifests, attestations, or other canonical artifacts capable of supporting claims.

Provenance frameworks such as W3C PROV-O model entities, activities, agents, and relationships such as generation, usage, derivation, and attribution (Lebo et al., 2013). CSE shares this concern with provenance, but adds standing and refusal. The evidence question becomes: what claim does this artifact support, under what authority, at what standing, with what non-claims preserved?

9.4 Clean Refactors That Damage Meaning

Some refactors improve implementation structure while damaging semantic structure. A developer may merge modules, rename fields, remove "redundant" status states, collapse receipts into logs, or move policy checks into UI code. The result may be cleaner by ordinary code metrics and worse by constitutional metrics.

This is improper semantic defragmentation: rearranging a system so it appears simpler while making its meaning less recoverable.

CSE therefore treats refactor as a semantic operation. It asks not only whether behavior was preserved, but whether state meaning, authority boundaries, refusal modes, receipt semantics, and standing claims were preserved.

10. Existing Practices and Their Limits

CSE is not born in a vacuum. It is a synthesis of long-standing software-engineering practices.

10.1 Modular Decomposition and Information Hiding

Parnas argued that modularization should be judged by the criteria used to divide a system, and that decompositions based on information hiding can improve flexibility and comprehensibility (Parnas, 1972). CSE extends this insight from implementation decisions to semantic decisions: a module should hide volatile implementation details, but it should not hide authority, standing, or claim boundaries.

10.2 Domain-Driven Design

Domain-Driven Design provides bounded contexts, ubiquitous language, and model-driven alignment between software and domain knowledge. Evans's work places domain meaning at the center of software design and emphasizes language as a binding force between model and implementation (Evans, 2003).

CSE accepts this but asks for stronger admissibility. A bounded context tells us where a model's language holds. CSE asks what authority permits transitions inside that context, what evidence proves those transitions, and what standing the resulting state has.

10.3 Hexagonal Architecture

Cockburn's hexagonal architecture, also called ports and adapters, aims to let an application be driven by users, programs, automated tests, or batch scripts while remaining isolated from runtime devices and databases (Cockburn, 2005).

CSE constitutionalizes ports. A port is not merely a technical seam. It is a possible authority boundary. An adapter is not merely infrastructure. It is a mediated world-contact surface. A test harness is not merely a driver. It is a candidate evidence generator.

10.4 Functional Core, Imperative Shell

The functional-core pattern separates deterministic computation from impure effects. CSE adopts this as a practical default: the core should produce semantic transitions and effect requests; adapters should mediate contact with the world. But CSE adds claim discipline. The important distinction is not only pure versus impure, but admissible versus inadmissible, authorized versus unauthorized, standing-preserving versus standing-smuggling.

10.5 Event Sourcing and Provenance

Event sourcing stores state changes as a sequence of events, enabling reconstruction of application state and history (Fowler, 2005). CSE treats events as evidence candidates, not automatically as evidence. An event becomes constitutionally useful when bound to state hashes, authority references, policy versions, refusal semantics, and explicit claims. The event tells what happened. The receipt tells why it can stand.

10.6 Architecture Decision Records

Architecture Decision Records preserve important architectural decisions, their context, and their consequences. The ADR movement is commonly associated with Michael Nygard's "Documenting Architecture Decisions" and later tooling and templates that make decision records part of development practice.

CSE extends decision records into standing records. A decision is not only architectural memory. It may be an authority transition: admit, refuse, defer, grant standing, deny standing, migrate standing, or preserve non-claim.

11. The Gap

Existing practices answer many questions well:

- DDD asks: what does the domain mean?
- Hexagonal architecture asks: where is the application boundary?
- Functional-core design asks: what is deterministic and what is effectful?
- Event sourcing asks: what changed?
- Provenance asks: what produced what?
- ADRs ask: why was this decision made?
- Capability security asks: what authority can access which resource?

CSE asks all of these, but under a unified constitutional question:

> What may this system now claim, under what authority, on what evidence, within what scope, after what admissible transition, with what refusals and non-claims preserved?

That is the gap. Without this question, systems often drift from behavior to confidence, from confidence to trust, and from trust to authority without an explicit standing transition.

12. Core Concepts

12.1 Semantic Locality

Semantic locality is the principle that every meaningful distinction should have a stable, recoverable home.

Examples:

- State shape belongs in a canonical model.
- Authority rules belong in policy surfaces.
- Effect boundaries belong in adapters or ports.
- Evidence belongs in receipts or manifests.
- Claims and non-claims belong in claim surfaces.
- Terms belong in a glossary.
- Standing transitions belong in decision records.

Loss of semantic locality creates semantic locality debt. The meaning still exists somewhere, but the system cannot recover it locally and canonically. It must infer meaning from scattered code, comments, non-binding traces, tests, conventions, and memory.

12.2 Authority

Authority is the declared basis on which a transition or effect may occur.

CSE distinguishes ability from authority. A function may be technically able to write a file, but unless the operation is authorized under the relevant context, policy, standing, and capability, the write is inadmissible.

A CSE-oriented system therefore passes authority explicitly:

```
Command + State + GovernedContext -> TransitionResult
```

The context should identify authority references, policy version, time source, input hashes, and any relevant capability or scope.

12.3 Effect Mediation

Effects are world-touching operations: filesystem writes, network calls, database mutations, process launches, compiler invocations, clock reads, randomness, external API calls, or substrate mutations.

CSE prefers that deterministic cores request effects rather than directly perform them. The effect request can then be inspected, admitted, refused, logged, or transformed by an adapter.

This creates a governable distinction:

```
meaningful transition requested an effect
governor or adapter admitted or refused the effect
receipt recorded the result
```

12.4 Receipt

A receipt is a structured record binding a transition to its evidence.

A minimal receipt should identify:

- receipt schema;
- operation;
- input hash;
- prior state hash;
- new state hash;
- authority reference;
- policy version;
- effects requested;
- effects admitted or refused;
- refusal modes;
- non-claims;
- artifact hashes;
- standing before and after, if applicable.

A non-binding trace says something happened. A receipt says what happened in a form that can support or refuse a claim.

12.5 Refusal

Refusal is a first-class outcome. In CSE, a refusal is not merely an error. It is a successful preservation of a boundary.

Examples:

- missing authority;

- noncanonical path;
- invalid standing transition;
- stale schema;
- effect outside allowed root;
- claim unsupported by evidence;
- execution attempted before standing exists.

A system that cannot refuse cannot govern.

12.6 Standing

Standing is the scoped status a package, artifact, transition, receipt, or subsystem has under a governance model.

Standing is not confidence. It is not mere test success. It is a governed semantic status.

Examples:

```
not_admitted_validation_only
package_admitted_fixture_scoped
governed_package_execution_standing_fixture_scoped
schema_conformant_receipt_standing
MCIR standing
equivalence standing
self-hosting standing
```

Standing must be granted, refused, deferred, or revoked by explicit transition records. It must not be inferred casually from adjacency.

12.7 Non-Claims

A non-claim is an explicit statement of what the system does not assert.

Examples:

```
This package is not yet Jnana-admitted.
This validation does not grant governed execution standing.
This receipt is not schema-conformant C-D.MS standing.
This fixture success is not a self-hosting claim.
This migration does not establish equivalence.
```

Non-claims are not disclaimers in the weak legal sense. They are semantic guardrails. They prevent implication from becoming unauthorized authority.

13. The CSE Application Pattern

A CSE-compatible application should be organized so a future governor can inspect it without reverse-engineering its meaning.

A practical structure is:

```
my_app/
  README.md
  GLOSSARY.md
  NONCLAIMS.md
  package_manifest.json
  src/
    my_app/
      core/
        model.py
        logic.py
        policy.py
      adapters/
        filesystem.py
        network.py
        subprocesses.py
        clock.py
        randomness.py
      receipts/
        schema.py
        emit.py
      cli.py
  tests/
    test_core.py
    test_policy.py
    test_refusals.py
    test_receipts.py
    test_golden_fixtures.py
  fixtures/
    input/
    expected/
  evidence/
    golden_runs/
  scripts/
    run_fixture.py
    verify.py
    clean_hygiene.py
```

The governing separation is:

```
core/      = deterministic meaning and policy
adapters/  = mediated side effects
receipts/  = evidence surfaces
fixtures/  = replay surfaces
tests/     = success and refusal checks
```

The core should express transitions. The adapters should touch the world. The receipts should bind what happened. The tests should prove both success and

refusal. The non-claims should prevent standing inflation.

14. Why Prefer CSE?

CSE is not necessary for every script. It is most useful when the cost of semantic drift is high: governed AI systems, compiler bootstrapping, self-hosting systems, audit-sensitive software, scientific workflows, election or civic infrastructure, financial systems, long-lived protocols, agent-built codebases, and systems expected to migrate across substrates.

You would prefer CSE because it provides capabilities that ordinary development practices do not usually provide together.

14.1 Governed Refactoring

CSE allows refactors to be checked for semantic preservation, not merely behavioral preservation.

The governing question becomes:

> Did the refactor preserve state meaning, authority boundaries, refusal modes, receipt semantics, standing claims, and non-claims?

This makes it possible to detect improper semantic defragmentation: cleanups that repay code debt by creating semantic debt.

14.2 Admission-Ready Software

CSE prepares software for future admission into stronger governance systems. A package with manifests, fixtures, receipts, refusal tests, authority surfaces, and non-claims can be evaluated without semantic archaeology.

The package already presents handles for governance.

14.3 Anti-Overclaim Architecture

CSE makes it difficult to smuggle authority through vague language.

It distinguishes:

```
validated != admitted
admitted != execution standing
execution standing != execution occurred
receipt exists != schema-conformant receipt standing
fixture success != broad maturity
equivalence fixture != self-hosting
self-produced evidence != independent ratification
bounded check != exact proof
```

This is an architecture against false ascent.

14.4 Semantic Observability

Modern observability says what happened operationally: a function ran, a request returned, a process exited, a file changed.

CSE adds semantic observability:

```
transition admitted
authority present
effect requested
effect mediated
receipt emitted
refusal preserved
standing changed
non-claims maintained
```

This is a different class of visibility.

14.5 Cross-Substrate Migration

When software moves from Python to C, from scripts to MCIR, from local execution to governed substrate, or from prototype to protocol, ordinary portability is insufficient. The system must preserve meaning.

CSE gives migration a checklist:

- Which states must remain equivalent?
- Which effects must remain mediated?
- Which authority references must survive?
- Which receipts must map?
- Which refusal modes must be preserved?
- Which standings are retained, lost, or newly earned?
- Which claims remain non-claims?

This is semantic portability.

14.6 Agent-Safe Development

AI coding agents can generate plausible code that silently changes meaning. CSE gives agents constraints:

- no new standing without decision record;
- no new acronym without glossary entry;
- no authority boundary change without policy update;
- no effect in core logic;
- no deletion of non-claims;
- no claim without evidence;
- no validation-to-admission leap;
- no passing-test-to-standing leap.

CSE therefore serves as a harness for agentic development.

15. CSE and Semantic Debt

CSE is the constructive counterpart to semantic debt.

Semantic debt has several subdomains:

- lexical debt: unclear or drifting terms;
- ontological debt: wrong or collapsed categories;
- boundary debt: meaning crosses contexts without translation;
- procedural debt: transitions and decisions lack semantics;
- evidentiary debt: meaning cannot be proven;
- authority debt: action lacks declared permission;
- claim debt: assertions outrun evidence.

Fragmentation is cross-cutting. Authority can be fragmented. Evidence can be fragmented. Claims can be fragmented. Terminology can be fragmented. The specific subtype most relevant to CSE is semantic locality debt: meaning exists, but not in a stable canonical home.

CSE repays semantic debt by restoring locality, boundary, authority, evidence, refusal, and standing.

16. Minimal CSE Profile

A minimal CSE-ready package should include:

```
[ ] deterministic core separated from adapters
[ ] typed state and command models
[ ] explicit governed context
[ ] authority surfaces named
[ ] side effects mediated through adapters
[ ] receipt schema
[ ] refusal vocabulary
[ ] success and refusal tests
[ ] replayable fixtures
[ ] package manifest
[ ] glossary
[ ] non-claims file
[ ] claim-to-evidence map
[ ] archive hygiene script
[ ] dependency lockfile
[ ] no hidden bytecode or cache artifacts in release archive
```

This profile is intentionally modest. It does not require a full formal methods stack. It does not require a finished MCIR substrate. It simply makes ordinary software legible to future governance.

17. CSE-Equivalence

CSE-equivalence, abbreviated CSE-Eq, is a semantic equivalence relation between two versions of a package or system.

Two versions V and V' are CSE-equivalent only if they preserve behavior and constitutional semantics.

17.1 Behavioral Equivalence

For all declared fixtures, outputs must match:

$$\forall fixtures,\ outputs_V = outputs_{V'}$$

This is necessary but not sufficient.

17.2 Policy Equivalence or Governed Policy Migration

Policies must be equivalent, or the policy migration must carry proof obligations:

$$\pi_V \equiv \pi_{V'}$$

or:

$$\pi_V \to \pi_{V'} \text{ with migration receipt and obligations}$$

A policy change that widens authority is not a harmless refactor.

17.3 Receipt Bisimulation

Receipts must be bisimilar, preserving binding fields except for allowed renames under an explicit mapping μ:

$$R_V \sim_\mu R_{V'}$$

Allowed renames must be declared. A receipt field cannot disappear silently if it contributes to admissibility, authority, standing, or claim support.

17.4 Standing Graph Preservation

The standing graph must be preserved up to isomorphism:

$$G_{st}^V \cong G_{st}^{V'}$$

If the standing graph changes, the change is a governed semantic migration, not an ordinary refactor.

CSE-Eq therefore defines a stronger refactor standard:

```
same output
same or governed policy
same receipt meaning
same standing graph
same non-claim force
```

18. Migration Contracts

CSE migration contracts support cross-substrate lift: for example, from Python to C, from scripts to MCIR, from local package to governed runtime, or from one receipt schema to another.

A migration contract defines mappings:

$$\mu_S,\ \mu_R,\ \mu_\pi$$

Where:

- μ_S maps source states to target states.
- μ_R maps source receipts to target receipts.
- μ_π maps source policy surfaces to target policy surfaces.

18.1 State Equivalence

The migrated state must preserve declared state meaning:

$$\mu_S(S) \approx S'$$

The relation $\approx$ must be defined by the migration contract. It may be exact equality, canonical equivalence, observational equivalence, or a domain-specific relation.

18.2 Receipt Preservation

Mapped receipts must retain binding fields:

$$\mathsf{bind}(\mu_R(R).S_{prev}, \mu_R(R).S_{next}, \mu_R(R).auth, \mu_R(R).pol, \mu_R(R).hashes)$$

A migrated receipt that loses authority, policy, state, or artifact binding cannot support the same claims.

18.3 Policy Refinement

The target policy must not widen authority without explicit authorization:

$$\pi' \sqsubseteq \pi$$

Here $\sqsubseteq$ denotes policy refinement: the target policy is at least as restrictive as the source policy with respect to protected actions, unless a widening is explicitly admitted by a migration record.

18.4 Dual-Hash Migration Receipts

A migration should produce receipts that bind both source and target artifacts:

```
source_state_hash
target_state_hash
source_receipt_hash
target_receipt_hash
source_policy_hash
target_policy_hash
migration_contract_hash
```

A cross-substrate lift without dual hashes risks semantic amnesia.

19. Minimal Tooling Interface

CSE can be supported by a minimal command-line interface. The following commands are illustrative rather than prescriptive.

19.1 cse check

Runs static invariants and schema validation.

Typical checks:

```
policy totality
authority plumbing
adapter isolation
glossary coverage
non-claim coverage
standing graph validity
receipt schema validity
package manifest consistency
```

19.2 cse replay

Runs fixture replay and emits receipt diffs.

Typical checks:

```
fixture output equivalence
receipt stability
normalized timestamp handling
refusal behavior
state hash consistency
```

19.3 cse claim

Evaluates the claim ledger.

Typical checks:

```
supported claims
blocked claims
unsupported claims
stale claims
claim-to-evidence satisfiability
```

19.4 cse diff

Produces a semantic diff.

Typical checks:

```
policy diff
standing graph diff
receipt schema diff
non-claim diff
glossary diff
authority surface diff
adapter boundary diff
```

19.5 cse pack

Produces an admission-ready artifact.

Typical outputs:

```
manifest
lockfile
glossary
non-claims
receipt schemas
fixtures
evidence chains
claim ledger
archive hygiene report
```

The tooling interface is intentionally small. Its purpose is to make CSE operational without requiring a finished constitutional runtime.

20. Worked Derivation: Admitted Claim and Blocked Claim

Consider a report-export operation.

20.1 Core Transition

The core receives:

```
S_prev
Ctx
Cmd = export_report(path, report_spec)
```

The core checks policy and emits an effect request rather than directly writing the file:

$$R_0 : eff_req = \{write(path, bytes)\}$$

20.2 Adapter Decision

The adapter or governing layer evaluates the effect request:

$$\pi(write, Ctx) = permit \Rightarrow eff_adm = \{write(path, bytes)\}$$

If the path is noncanonical, the effect may instead be refused:

$$\pi(write, Ctx) = refuse(noncanonical_path)$$

20.3 Final Receipt

The final receipt includes:

```
op = export_report
S_prev = hash(S_prev)
S_next = hash(S_next)
auth_ref = A_7
pol_ver = 3.2
eff_req = {write(path, bytes)}
eff_adm = {write(path, bytes)}
eff_ref = {}
ref_codes = {}
nc_refs = {no_equivalence, no_self_hosting,
no_schema_conformant_standing}
```

20.4 Admitted Claim

The claim:

```
export_admitted
```

is admissible when the claim ledger can derive:

$$\begin{aligned}&\exists R : R.op = export_report\\&\wedge R.eff_adm\\&\neq \varnothing\\&\wedge \pi(write, Ctx) = permit\\&\wedge \mathsf{adm}(R)\end{aligned}$$

If these conditions hold and no active non-claim blocks `export_admitted`, then `export_admitted` belongs to the admissible claim frontier:

$$export_admitted \in \mathcal{K}_{adm}$$

20.5 Blocked Claim

The claim:

`schema_conformant_standing`

does not follow merely because a receipt exists. It holds only if the receipt conforms to the relevant schema and a standing transition record grants that standing:

$$\begin{aligned}&schema(R) = v0\\&\wedge check(R) = true\\&\wedge \exists R_{st} : \mathsf{adm}(R_{st})\\&\wedge \delta_{st}(st, R_{st}) = schema_conformant_standing\end{aligned}$$

If the active non-claims include:

`no_schema_conformant_standing`

then:

$$NC \dashv schema_conformant_standing$$

and therefore:

$$schema_conformant_standing \notin \mathcal{K}_{adm}$$

The claim is blocked until an admissible record revokes the non-claim and grants the standing. This is the core CSE distinction: receipt existence is not standing; schema validation is not admission; execution is not maturity.

21. Improper Semantic Defragmentation

CSE clarifies why some cleanups are harmful.

A cleanup is suspicious when it:

- merges policy and execution;
- deletes refusal tests;
- replaces receipts with non-binding traces;
- removes non-claims;
- renames standings without migration;
- collapses distinct statuses into a generic `status`;
- moves side effects into core logic;
- treats test success as admission;
- treats admission as execution authority;
- updates a roadmap without updating the index;
- adds acronyms without glossary entries.

Such changes may reduce visible complexity while increasing semantic debt. CSE calls this improper semantic defragmentation.

Proper semantic defragmentation does the opposite. It moves meaning toward stable homes, clarifies boundaries, binds evidence, separates statuses, names refusals, and preserves non-claims.

22. Open Problems

CSE is a proposed discipline, not a completed formal system. Several problems remain open.

22.1 Compositional Standing Across Federated Systems

How should standing compose when multiple systems, each with its own policies and receipts, participate in one larger transition? The problem is not merely distributed consensus. It is distributed semantic admissibility. Section 5.8 sharpens the question rather than answering it: federated composition must also establish that the participants occupy genuinely distinct control domains, since a federation of nominally separate systems under one controller composes standing without composing independence.

22.2 Cryptographic Attestation Standards for Receipts

Receipts can be content-addressed and signed, but CSE requires conventions for what is signed, by whom, under what authority, and with what revocation or freshness semantics.

22.3 Policy Diff Proofs at Scale

Policies must be diffable, but large policies may require proof obligations to show that a change narrows, preserves, or widens authority. Practical tools are needed for policy-diff classification.

22.4 Automated Non-Claim Synthesis

A system may infer non-claims from test gaps, missing evidence, unproven migrations, unsupported standings, or unresolved schema changes. Automated NC synthesis could prevent claim inflation, but risks both overblocking and underblocking.

22.5 Category-Theoretic Model of Standing Transitions

Standing transitions may admit a categorical model, with systems as objects, admissible transitions as morphisms, and standing transformations as functorial structure over state categories. This remains speculative, but may offer a rigorous account of compositional standing and migration.

22.6 Control-Model Declaration and Verification

Section 5.8 depends on a declared control model, but declaring one is easier than verifying it. Which control channels must a declaration enumerate, how should shared infrastructure degrade an independence claim, how should correlated failure affect standing, and can control separation itself be attested rather than asserted? A control model that is merely stated by the subject inherits the problem it was introduced to solve.

22.7 Human Factors and Developer Adoption

CSE must avoid becoming ceremony. Its tooling and documents must be light enough that developers will use them before a crisis. The discipline succeeds only if it improves ordinary development while preserving future admissibility.

23. Relationship to Jnana and MCIR

CSE can be practiced before Jnana, MCIR, or any constitutional runtime is complete.

This is important. Developers can write ordinary Python applications today in a way that makes them easier to adapt later. The goal is not to imitate unfinished internals. The goal is to preserve semantic handles:

- deterministic core transitions;
- effect requests;
- authority contexts;
- receipts;

- refusal vocabularies;
- fixtures;
- non-claims;
- glossary entries;
- manifest standing.

When Jnana or MCIR later provides stronger admission, representation, execution, or equivalence machinery, CSE-ready applications will be easier to lift. They will already distinguish what they mean from what they do to the world.

The invariant of Section 5.8 has a more developed treatment elsewhere. A companion manuscript develops anti-self-ratification as a constitutional theory of legitimate state transition in reflexive systems, including the closure conditions under which the invariant cannot be circumvented by rewriting the rules that define it, the independence-rank machinery that resists nominal plurality, the liveness complement that prevents a fail-closed system from becoming permanently entrenched, and the boundary between amending an order and replacing it (Ableman, 2026). A governance kernel has separately realized much of that machinery under construction discipline, including exact and bounded anti-self-ratification search, control-domain declarations, conjunct-ablation courts that demonstrate each closure term is load-bearing, and anti-entrenchment reachability analysis.

The relation between the two is a division of labor rather than an overlap. This paper is the engineering discipline: make meaning, authority, evidence, refusal, standing, and non-claims recoverable across change. The companion is the constitutional theory: establish under what conditions a system may confer standing on itself at all. A package can be CSE-conforming without the second, and the second is difficult to realize in software that lacks the first.

24. Conclusion

Constitutional Semantic Engineering begins from a skeptical premise: running code is not enough. A system may execute while its meaning fragments, its authority assumptions hide, its evidence remains inadmissible, and its claims drift beyond what its artifacts can support.

Existing software practices offer powerful partial remedies. Domain-Driven Design protects language. Hexagonal architecture protects boundaries. Functional-core patterns protect determinism. Event sourcing and provenance protect reconstructability. ADRs protect decision memory. Capability systems protect authority. Testing protects behavior.

CSE composes these into a constitutional discipline.

It asks software to preserve not only behavior but standing; not only non-binding traces but receipts; not only errors but refusals; not only claims but non-claims; not only modularity but semantic locality; not only execution but admissibility.

The result is software that can be governed without being rewritten, migrated

without semantic amnesia, refactored without invisible authority loss, and admitted without relying on human memory.

The code may run. CSE asks whether its meaning can still stand.

Corpus-Wide Reference Positioning

This paper argues that software systems often remain operational long after their meaning has begun to decay. Its nearest external comparison fields are semantic, provenance, software. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Formal vocabularies and graph links make commitments explicit but cannot by themselves establish correct identity, shared interpretation, or lossless translation between contexts. A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Richard Cyganiak, David Wood, and Markus Lanthaler, eds. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 25 February 2014. https://www.w3.org/TR/rdf11-concepts/ **Relationship:** Adjacent graph data model and identity baseline.

[**E2**] Alistair Miles and Sean Bechhofer, eds. *SKOS Simple Knowledge Organization System Reference*. W3C Recommendation, 18 August 2009. https://www.w3.org/TR/skos-reference **Relationship:** Adjacent standard for concepts, labels, schemes, notes, and semantic mappings.

[**E3**] W3C OWL Working Group. *OWL 2 Web Ontology Language Document Overview*, Second Edition. W3C Recommendation, 11 December 2012. https://www.w3.org/TR/owl2-overview/ **Relationship:** Adjacent formal ontology standard used to compare declared semantics and inference commitments.

[**E4**] Mark D. Wilkinson et al. "The FAIR Guiding Principles for Scientific Data Management and Stewardship." *Scientific Data* 3, 2016, 160018. DOI: 10.1038/sdata.2016.18. https://doi.org/10.1038/sdata.2016.18 **Relationship:** Predecessor stewardship principles for findability, accessibility, interoperability, and reuse.

[**E5**] Peter Gardenfors. *Conceptual Spaces: The Geometry of Thought*. MIT Press, 2000. mitpress.mit.edu source **Relationship:** Predecessor geometric treatment of concepts; cited as comparison, not as validation of the paper's manifold terminology.

[**E6**] W3C. *PROV-O: The PROV Ontology*. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E7**] Santiago Torres-Arias et al. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 2019, pp. 1393-1410. usenix.org source **Relationship:** Predecessor supply-chain lineage system used for comparison; no influence claim is made unless the source head already documents one.

Ableman corpus

[**A1**] Adam Ableman. *Meaning as Binding*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-5F976917F0B283EA`;
source SHA-256
`A470D1A0B9ED7C3CEAB6BB88BE4B642A2BA36427447F787764329FD574DA0C97`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Governed Meaning Base (GMB).* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-D95CCC4FB7B58BFB;
source SHA-256
A3B62E50F8BEE125FDAE0D38C2839AFD1CCC138296268BF93217481E38AA48EA.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Semantic Manifold Topology.* Ableman Corpus current edition; work identity PAPER.PUB.20257244;
source SHA-256
B8F28CC652A73A3EE4CF6FD0E23C1ABC922A6660319B2FB4CC1CCAAA606EC64A.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Global Term Key: Identity Primitives in Governed Knowledge Systems.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-358AC09F5A13587E;
source SHA-256
AFB29CFCB4B58A78B79709AD453CEE9088CCB2AE40F30A70260F1B54F5160579.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Semantic Legitimacy Before Context.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-091E34BA93992CE4;
source SHA-256
DEB7626CE1AB54F4B541347DCC3C4E14DC4F682230ACCCE1A93D1D536A50C44C.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Reasoning by Way of Receipts.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-04E427F6F4D94454;
source SHA-256
AF86A66CCD6A103DD75208500AE99148911A6453B6DFDF36A954279F21AB8489.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *The Epistemic Lineage Engine.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-E0EC3DC7EABF5060;
source SHA-256
CDD74D9AFAB6532645E68F7F6C9F0F7A9E472D9B8E717290522566FF8DC01F11.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Cockburn, A. (2005). Hexagonal architecture: The original 2005 article. HaT Technical Report 2005.02. https://alistair.cockburn.us/hexagonal-architecture

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.

Fowler, M. (2005). Event sourcing. MartinFowler.com. https://martinfowler.com/eaaDev/EventSourcing.html

Kruchten, P., Nord, R. L., and Ozkaya, I. (2012). Technical debt: From metaphor to theory and practice. IEEE Software, 29(6), 18-21. DOI: 10.1109/MS.2012.167.

Lebo, T., Sahoo, S., McGuinness, D., Belhajjame, K., Cheney, J., Corsar, D., Garijo, D., Soiland-Reyes, S., Zednik, S., and Zhao, J. (2013). PROV-O: The PROV Ontology. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/

Miller, M. S. (2006). Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control. Doctoral dissertation, Johns Hopkins University. http://www.erights.org/talks/thesis/

Adam Ableman (2026). Legitimacy Without Self-Ratification: Constitutional Computing and the Provenance of Authority in Reflexive Systems. Manuscript.

Miller, M. S., Yee, K. P., and Shapiro, J. S. (2003). Capability myths demolished. Systems Research Laboratory, Johns Hopkins University. http://srl.cs.jhu.edu/pubs/SRL2003-02.pdf

Nygard, M. (2011). Documenting architecture decisions. Cognitect Blog. cognitect.com source

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. Communications of the ACM, 15(12), 1053-1058. DOI: 10.1145/361598.361623.

Storey, M. A. (2026). From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI. arXiv. https://arxiv.org/abs/2603.22106

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.21111455